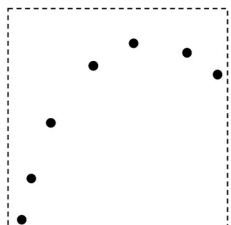
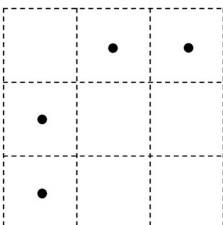
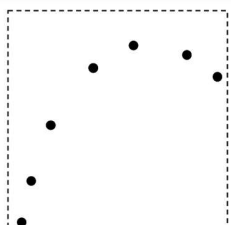
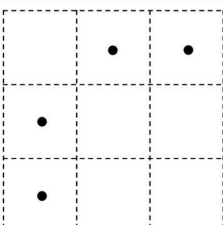


문제 1. 3D point cloud 를 처리하는 대표적인 방식인 **Point-based representation** 과 **Voxel-based representation** 을 비교하시오. 각 방식의 특징을 inference speed, accuracy, memory usage 관점에서 서술하시오.

	Point-based	Voxel-based
Data Representation		

답.

	Point-based	Voxel-based
Data Representation		
Inference Speed	Slow	Fast
Accuracy	Accurate	Inaccurate
Memory Usage	Heavy	Light

Point-based representation 은 point cloud 의 원래 좌표를 직접 사용하여 각 point 의 feature 를 처리하는 방식이다. 점의 세밀한 위치 정보를 보존할 수 있기 때문에 상대적으로 정확한 geometry 표현이 가능하고, 높은 accuracy 를 기대할 수 있다. 하지만 point 간 이웃을 찾기 위해 kNN search 나 grouping 연산이 필요하므로 inference speed 가 느릴 수 있으며, 많은 point 를 직접 처리해야 하므로 memory usage 도 큰 편이다.

Voxel-based representation 은 3D 공간을 일정한 grid 또는 voxel 로 나눈 뒤, point 들을 해당 voxel 에 할당하여 처리하는 방식이다. voxel grid 를 사용하면 convolution 이나 hash-based neighbor search 를 효율적으로 적용할 수 있어 inference speed 가 빠르고, sparse voxel 만 저장하면 memory usage 도 상대적으로 가볍다. 하지만 여러 point 가 하나의 voxel 로 quantization 되면서 세밀한 좌표 정보가 손실될 수 있으므로 point-based 방식에 비해 accuracy 가 낮아질 수 있다.

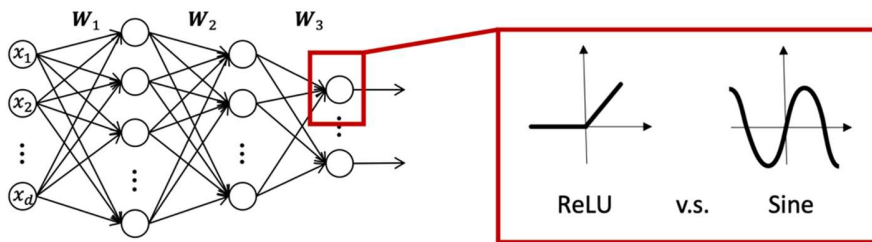
문제 2. 3D 데이터에서 일반적인 **dense convolution** 을 적용하기 어려운 이유를 설명하고, 이를 해결하기 위한 **Sparse Convolution** 의 기본 원리와 수행 방식을 서술하시오.

답.

3D point cloud 나 LiDAR 데이터는 대부분의 공간이 비어 있는 **sparse** 한 데이터이다. 따라서 전체 3D 공간을 **dense voxel grid** 로 만든 뒤 일반적인 3D **convolution** 을 적용하면, 실제 **point** 가 존재하지 않는 빈 공간까지 모두 계산해야 하므로 메모리 사용량과 연산량이 매우 커진다. 특히 3D **convolution** 은 해상도가 증가할수록 계산량이 급격히 증가하기 때문에, 대규모 3D **scene** 에 그대로 적용하기 어렵다.

Sparse Convolution 은 이러한 문제를 해결하기 위해, 전체 **voxel grid** 가 아니라 **point** 가 존재하는 **occupied voxel** 에 대해서만 **convolution** 을 수행한다. 먼저 3D **point** 들을 **voxel coordinate** 로 **quantization** 하고, **occupied voxel** 의 좌표와 **feature** 만 **sparse tensor** 형태로 저장한다. 이후 **hash table** 또는 **coordinate map** 을 이용해 각 **occupied voxel** 주변의 이웃 **voxel** 을 빠르게 찾고, 실제로 존재하는 **voxel feature** 에 대해서만 **convolution kernel** 을 적용한다.

문제 3. 다음은 **sine** 함수를 **activation** 으로 활용하는 **multi-layer perceptron** 의 그림을 나타내고 있다. 이 **MLP** 를 이용하여 **gray scale** 이미지를 **per-scene optimization** 하고자 하는 경우, **input** 은 무엇이 되고, **output** 은 무엇이 되는지 설명하시오.



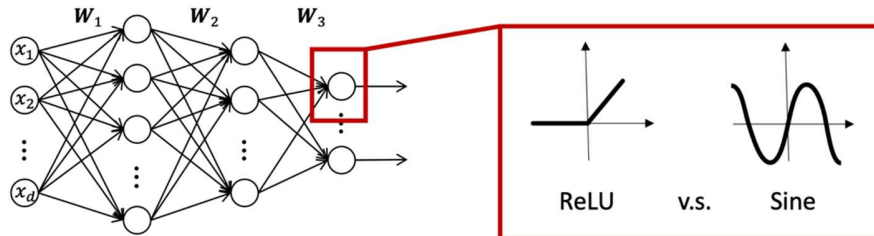
$$\Phi(\mathbf{x}) = \mathbf{W}_n (\phi_{n-1} \circ \phi_{n-2} \circ \dots \circ \phi_0)(\mathbf{x}) + \mathbf{b}_n,$$

$$\mathbf{x}_i \mapsto \phi_i(\mathbf{x}_i) = \boxed{\sin}(\mathbf{W}_i \mathbf{x}_i + \mathbf{b}_i)$$

$$\begin{aligned} \phi_i : \mathbb{R}^{M_i} &\mapsto \mathbb{R}^{N_i} & \mathbf{x}_i &\in \mathbb{R}^{M_i} \\ \mathbf{W}_i &\in \mathbb{R}^{N_i \times M_i} & \mathbf{b}_i &\in \mathbb{R}^{N_i} \end{aligned}$$

답. 입력 (x, y) coordinate, 출력 pixel intensity.

문제 4. 아래 그림에서 sine 함수를 ReLU 함수 대신 사용하면 좋은 점이 무엇인지 설명하시오.



$$\Phi(\mathbf{x}) = \mathbf{W}_n (\phi_{n-1} \circ \phi_{n-2} \circ \dots \circ \phi_0)(\mathbf{x}) + \mathbf{b}_n,$$

$$\mathbf{x}_i \mapsto \phi_i(\mathbf{x}_i) = \boxed{\sin}(\mathbf{W}_i \mathbf{x}_i + \mathbf{b}_i)$$

$$\begin{aligned} \phi_i : \mathbb{R}^{M_i} &\mapsto \mathbb{R}^{N_i} & \mathbf{x}_i &\in \mathbb{R}^{M_i} \\ \mathbf{W}_i &\in \mathbb{R}^{N_i \times M_i} & \mathbf{b}_i &\in \mathbb{R}^{N_i} \end{aligned}$$

답. $\sin(\mathbf{W}\mathbf{x}+\mathbf{b})$ 의 형태이기 때문에 자연스럽게 다차원 함수의 주기와 바이어스를 학습하게 됨. 여러 레이어를 쌓게 되면 데이터를 최적으로 표현할 수 있는 **basis function**들의 조합으로 함수를 표현할 수 있음. 따라서 **real-world data** (음성, 이미지, 비디오 등)의 처리가 훨씬 용이해짐

문제 5. 가우시안 스플래팅 관련 문제

가우시안 스플래팅은 썬 별 최적화 방식으로 시작했지만, 최근에는 **Feed-forward** 방식으로도 만들어진다. 더 나아가, 여러 이미지가 아닌 한 이미지로부터 가우시안 집합을 생성하는 연구들도 발표되고 있다. 다른 공간 표현방식이 아니라, 가우시안을 사용하는 이유가 무엇인가?

답.

- 사진과 같이 사실적 렌더링(**Photorealistic rendering**)이 가능하기 때문에
- 뉴럴 네트워크/뉴런/implicit representation/NeRF 등과 비교하여 용량이 가벼워 빠르기 때문에
- 해당 표현형이 **explicit** 하기 때문에